

Taskify — Frontend Implementation Plan

Day-wise development roadmap based on the provided BRD, FRD, HLD, LLD, API Specification, Database Design, and the current frontend folder structure.

Working assumption: 5 focused hours/day with AI-assisted (vibe) coding. Existing frontend functionality will be audited and reused rather than rewritten from scratch.

1. Project Context

Frontend target: React + Vite + PWA. The overall system uses Node.js + Express.js + MongoDB/Mongoose and a modular-monolith architecture.

Primary roles: SUPER_ADMIN, ADMIN / DEPARTMENT HEAD, EMPLOYEE.

Frontend principle: Page → Feature Components → Hooks/State → API Service → Backend API.

Migration principle: Keep working existing code and migrate incrementally instead of rewriting the whole project.

2. Current Frontend Structure

```
client/src/
■■■ main.jsx
■■■ App.jsx
■■■ components/layout/
■■■ pages/auth/
■■■ pages/superadmin/
■■■ pages/admin/
■■■ pages/employee/
■■■ services/api.js
■■■ utils/notifications.js
```

Initial assessment: Authentication, dashboards, task assignment, task views/status, reports, notifications and profile-related screens already appear to exist from the folder structure. Their actual working status must be verified from the code before marking them complete.

3. Target Frontend Direction

```
src/
■■■ app/
■■■ components/
■■■ features/
■ ■■■ auth/
■ ■■■ dashboard/
■ ■■■ tasks/
■ ■■■ users/
■ ■■■ departments/
■ ■■■ notifications/
■ ■■■ reports/
■■■ layouts/
■■■ pages/
■■■ routes/
■■■ services/
■■■ hooks/
■■■ utils/
■■■ constants/
■■■ validators/
■■■ context/
```

Important: Do not create/move everything blindly on Day 1. First audit the existing implementation, then migrate feature by feature.

DAY 1 — Existing Frontend Audit

- Do not make major architecture changes yet.
- Trace `main.jsx` → `App.jsx` → routing → `ProtectedRoute` → role → page.
- Inspect authentication state, token storage, current-user handling and role checks.
- Inspect `services/api.js` and identify whether API calls are centralized or written directly inside components.
- Inspect `auth`, `superadmin`, `admin` and `employee` pages and map each page to its API/component dependencies.
- Create an audit table: KEEP, MODIFY, REFACTOR, DELETE, CREATE.

5-hour focus: 5-hour focus: 1h entry/routing, 1h layout/auth guard, 1h API service, 1h pages, 1h audit checklist.

DAY 2 — Frontend Architecture Setup

- Introduce the target feature-oriented structure gradually.
- Separate page, feature component, hook/state and API-service responsibilities.
- Do not delete working files simply because their location is old.
- Begin moving duplicated task/report logic toward feature modules.
- Prepare the base for feature-specific services such as `authApi`, `userApi`, `departmentApi` and `taskApi`.

5-hour focus: Goal: establish clean responsibility boundaries without breaking existing functionality.

DAY 3 — Authentication + Authorization

- Verify/refactor Login, Logout and Current User flow.
- Implement/verify protected routes.
- Implement/verify role-based routes for `SUPER_ADMIN`, `ADMIN` and `EMPLOYEE`.
- Verify authentication state persistence and redirect behavior.
- Verify change-password, forgot-password and reset-password screens if included in the current scope.

5-hour focus: Core flow: Login → Auth API → Token/Session → Current User → Role → Protected Route → Dashboard.

DAY 4 — Common Layout + Navigation

- Header, Sidebar and common layout.
- Role-based navigation menus.
- Profile access and logout.
- Notification bell entry point.
- Breadcrumbs where needed.
- Loading, error and empty-state UI.

5-hour focus: Goal: common shell should work consistently for all three roles.

DAY 5 — Users Management

- Users list.
- Create user.

- Edit user.
- View user.
- Activate/deactivate user.
- Search, filter and pagination.
- Reuse existing AdminList/EmployeeList/AdminEmployeeList where they are working; consolidate duplicated behavior.

5-hour focus: Relevant API surface: GET/POST/PATCH user endpoints and user status/tasks endpoints.

DAY 6 — Departments

- Department list.
- Create department.
- Edit department.
- Department details.
- Assign/change Department Head.
- Activate/deactivate department.
- Show department employees where required.

5-hour focus: Relevant API surface: GET/POST/PATCH department endpoints.

DAY 7 — Task List

- All Tasks and My Tasks.
- Search.
- Status filter.
- Priority filter.
- Department filter.
- Assignee filter.
- Due-date filters.
- Sorting.
- Pagination.

5-hour focus: Use server-side filtering/pagination through the task API; do not download all tasks and filter them in React.

DAY 8 — Create/Edit Task + Task Details

- Create Task form.
- Edit Task form.
- Task details page.
- Title, description, priority and due date.
- Department and assignee selection.
- Prepare the UI for assignments, subtasks, history, comments and attachments.

5-hour focus: Core flow: Create Task → Select Department → Select Employee(s) → Priority → Due Date → Submit.

DAY 9 — Task Assignment + Employee My Tasks

- Implement/refactor assignment UI.
- Support multiple assignees where required.
- Show task assignments separately from the task itself.
- Employee My Tasks should use the task-assignment concept.
- Assignment detail view.
- Reassignment UI where permitted.

5-hour focus: Relevant APIs include task assignment creation/listing, My Tasks, assignment detail, status update and reassignment.

DAY 10 — Task Status + Task History

- Status actions for permitted users.
- ASSIGNED → ACKNOWLEDGED → IN_PROGRESS → COMPLETED.
- Support BLOCKED flow if enabled by final rules.
- Show/disable status actions according to current state.
- Task history timeline.
- Keep history read-only from the normal frontend.

5-hour focus: Important: UI can guide valid transitions, but final authorization and transition validation belong to the backend.

DAY 11 — Subtasks + Progress

- Create subtask UI.
- Subtask list inside Task Details.
- Subtask assignment.
- Subtask status display.
- Progress summary.
- Parent task progress display.

5-hour focus: Relevant APIs: POST /api/tasks/:taskId/subtasks, GET subtasks, GET progress.

DAY 12 — Role-based Dashboards

- Super Admin dashboard.
- Admin/Department dashboard.
- Employee dashboard.
- Summary cards.
- Task metrics.
- Recent activity.
- Overdue/upcoming tasks.
- Reports/quick actions as required.

- Connect dashboard to summarized dashboard API.

5-hour focus: Do not calculate large dashboard datasets in React; use the dashboard/report API.

DAY 13 — Notifications + Profile

- Notification bell.
- Unread count.
- Notification list.
- Mark one as read.
- Mark all as read.
- Notification-to-task navigation.
- Profile page.
- Allowed profile editing.
- Change password entry point.

5-hour focus: Relevant notification APIs: list, unread-count, mark-read and read-all.

DAY 14 — Reports

- Task reports.
- Employee performance.
- Department performance.
- My performance.
- Date/status/priority/department/assignee filters where applicable.
- Role-specific visibility.
- Charts/tables/export UI only where supported by the finalized requirements.

5-hour focus: Use report APIs; backend is responsible for scope and report calculations.

DAY 15 — Recurring Tasks + Comments

- Recurring task list.
- Create/edit recurring definition.
- Pause/resume/cancel actions.
- DAILY/WEEKLY/MONTHLY options if finalized.
- Comments list/create/edit/delete only if the final business rules include them.
- Keep comments associated with the task and author.

5-hour focus: Some recurring-task and comment business rules are explicitly marked for finalization, so do not invent behavior.

DAY 16 — Attachments + PWA + UI Polish

- Attachment upload/list/view/delete UI.
- Respect authorization-driven file access.
- Validate user-facing file constraints when finalized.

- PWA setup/verification.
- Responsive/mobile UI.
- Loading states.
- Error states.
- Empty states.
- Form validation and UI consistency.

5-hour focus: PWA is part of the frontend requirement. File authorization must ultimately be enforced by the backend.

DAY 17 — Full Frontend Integration Testing / Buffer

- Super Admin creates Admin → Admin login.
- Admin creates task → assigns employee → employee sees task.
- Employee changes status → history generated → relevant notification.
- Main task → subtasks → progress.
- Role-based access checks.
- Search/filter/pagination checks.
- Notifications and reports checks.
- Responsive/PWA checks.
- Fix remaining frontend bugs.

5-hour focus: Use this as a testing day and buffer. If the previous days finish early, start integration testing earlier.

4. Frontend Feature Checklist

Feature	Main Frontend Work	Target
Authentication	Login, logout, current user, protected/role routes	P0
Users	List, create, edit, activate/deactivate, filters	P0
Departments	CRUD/status/head management	P0
Tasks	List, create, edit, details, filters, pagination	P0
Assignments	Assign, My Tasks, status, reassign	P0
Task History	Read-only timeline/history	P0
Dashboard	Role-scoped summarized metrics	P0
Notifications	List, unread count, read/read-all	P1
Subtasks	Create/list/progress	P1
Reports	Task and performance reports	P1
Recurring Tasks	Create/list/edit/pause/resume/cancel	P1
Comments	Create/list/edit/delete if finalized	P1
Attachments	Upload/list/view/delete	P1
WhatsApp	Mostly backend/integration; frontend only where UI is needed	P1
PWA	Installable responsive React experience	Requirement

5. Estimated Timeline

Frontend: approximately 12–16 working days in the earlier estimate; after reviewing the supplied folder structure, a practical working plan is **15–17 days** including migration, missing features and testing.

5 hours/day: approximately 75–85 focused hours for the frontend plan.

Backend + integration: roughly 15–20 days backend plus 5–7 days integration/testing/deployment was estimated earlier. This should be refined after the current backend code is audited.

6. Vibe Coding Rules

Do not treat AI-generated code as automatically correct. For every feature use this cycle:

```
Requirement
↓
Existing Code Check
↓
API Contract
↓
Frontend Implementation
↓
Run
↓
Test
↓
Fix
↓
Mark Complete
```

Do not invent new endpoints when the API specification already defines the contract. If an API appears missing, flag it before creating it.

7. Important Open Decisions

The supplied documents explicitly leave some items for finalization. Do not guess these during implementation:

- Exact task status transition matrix.
- Exact permission matrix.
- Exact request/response schemas.
- Exact WhatsApp provider and interaction format.
- Email/SMS providers.
- Exact recurring-task rules.
- Attachment limits.
- Some comment business rules.

These decisions should be confirmed before implementing behavior that depends on them.

8. Immediate Next Step

Start with Day 1 only. Provide/inspect the actual client source code, especially package.json, App.jsx, main.jsx, services/api.js, routing/auth files, and the existing page/component files. Then mark every feature as KEEP, MODIFY, REFACTOR, DELETE or CREATE.

The folder structure alone is not enough to declare a feature working. Actual code and runtime behavior must be checked.